AIML-Assignment 4

1. Write a Python program to draw a line with suitable labels for the X-axis and Y-axis and a suitable title.

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 6, 8, 10]

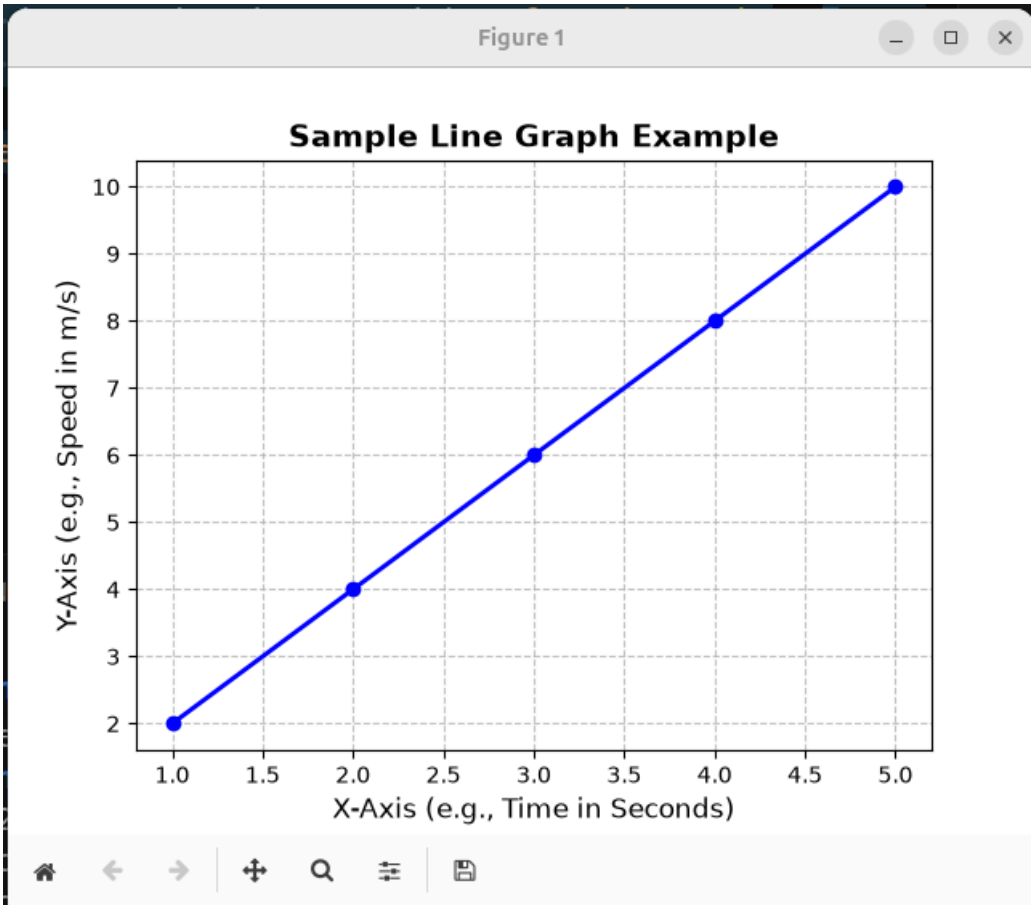
plt.plot(x_values, y_values, marker='o', color='blue', linestyle='-', linewidth=2)

plt.title('Sample Line Graph Example', fontsize=14, fontweight='bold')
plt.xlabel('X-Axis (e.g., Time in Seconds)', fontsize=12)
plt.ylabel('Y-Axis (e.g., Speed in m/s)', fontsize=12)

plt.grid(True, linestyle='--', alpha=0.7)

plt.show()
```

Output:



2. Write a Python program to draw a line using the X-axis and Y-axis values taken from a text file, with suitable labels for the X-axis and Y-axis and a suitable title.

test.txt

1 2
2 4
3 1

Source code:

```
import matplotlib.pyplot as plt

x_values = []
y_values = []

with open('test.txt', 'r') as file:
    for line in file:
        parts = line.split()
        if len(parts) == 2:
            x_values.append(float(parts[0]))
            y_values.append(float(parts[1]))

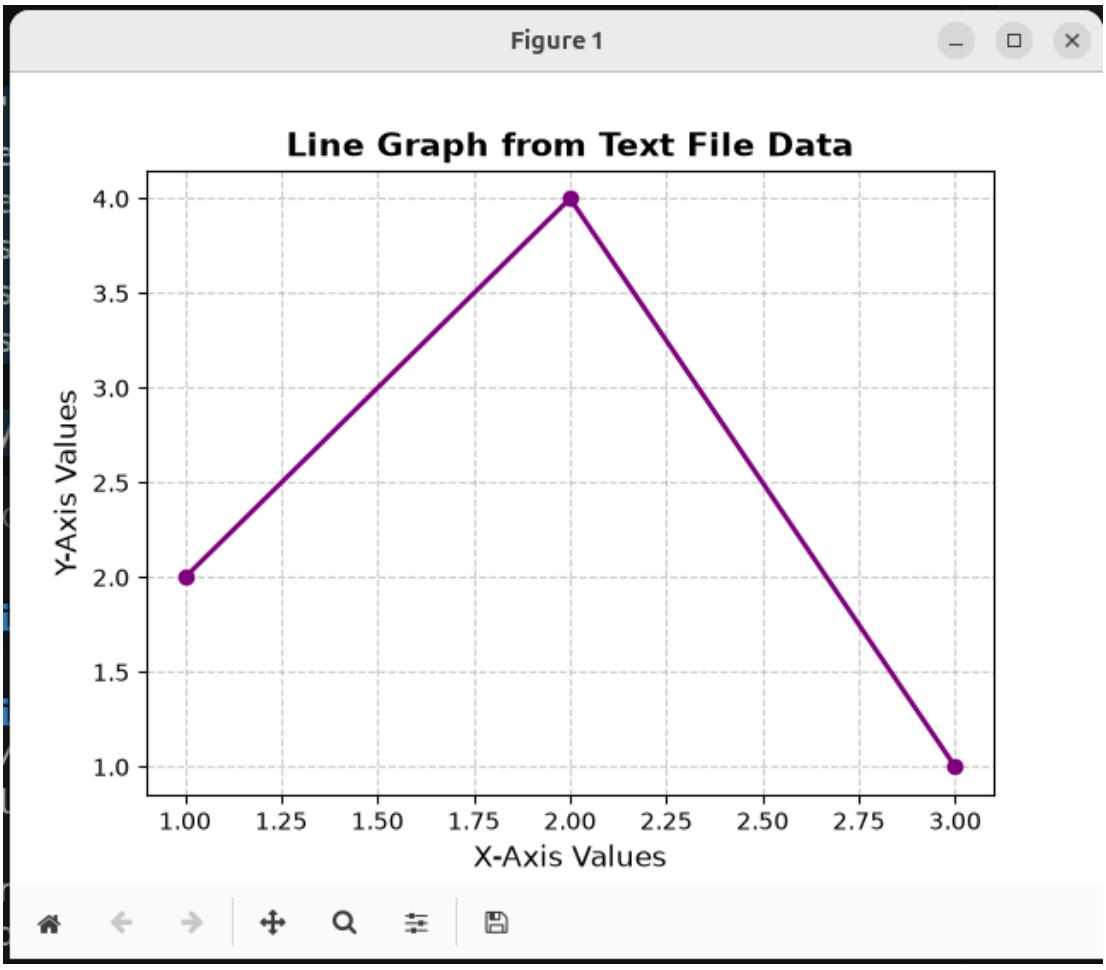
plt.plot(x_values, y_values, marker='o', color='purple', linestyle='-', linewidth=2, markersize=6)

plt.title('Line Graph from Text File Data', fontsize=14, fontweight='bold')
plt.xlabel('X-Axis Values', fontsize=12)
plt.ylabel('Y-Axis Values', fontsize=12)

plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```

Output:



3. Write a Python program to draw line charts for the financial data of Alphabet Inc. from October 3, 2016 to October 7, 2016.

Date,Open,High,Low,Close

10-03-16,774.25,776.065002,769.5,772.559998
10-04-16,776.030029,778.710022,772.890015,776.429993
10-05-16,779.309998,782.070007,775.650024,776.469971
10-06-16,779,780.47998,775.539978,776.859985
10-07-16,779.659973,779.659973,770.75,775.080017

Source Code:

```
import io
import pandas as pd
```

```
import matplotlib.pyplot as plt

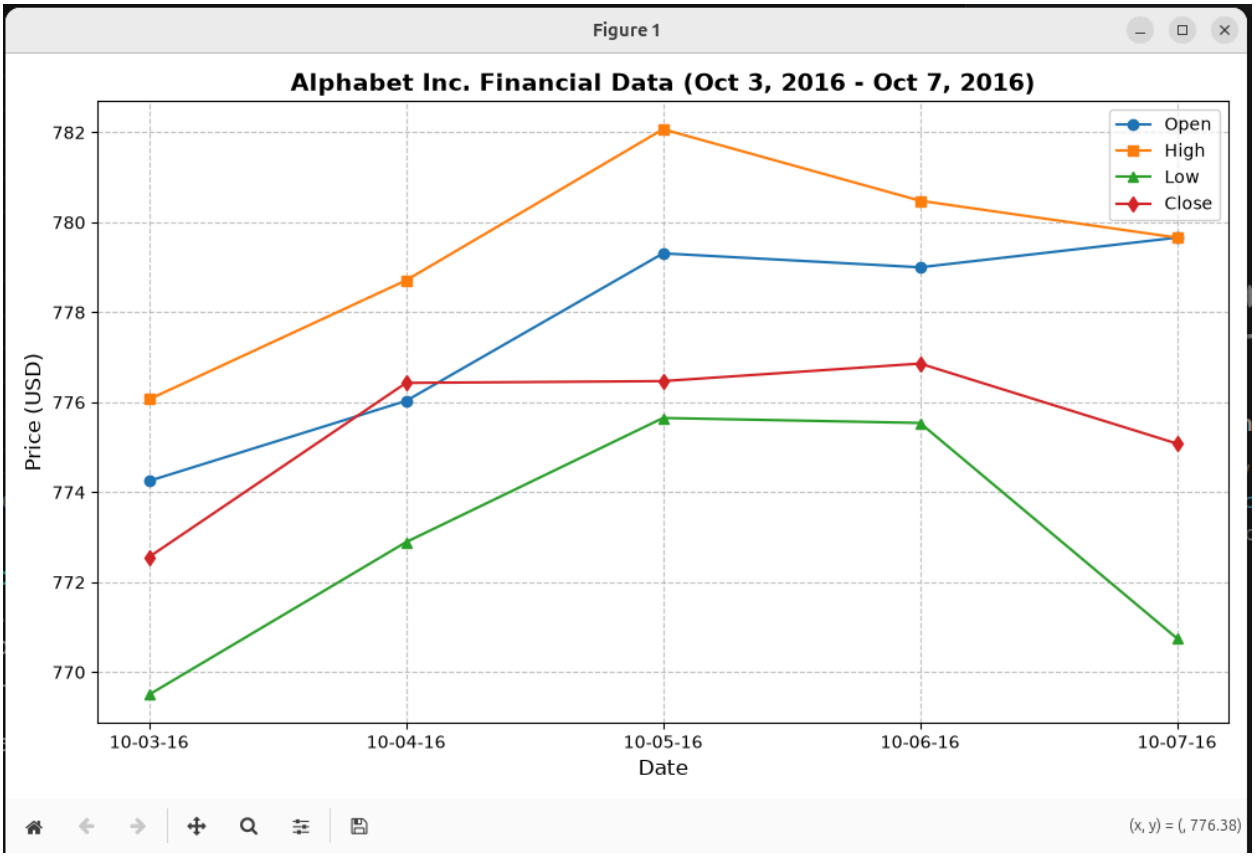
data = """Date,Open,High,Low,Close
10-03-16,774.25,776.065002,769.5,772.559998
10-04-16,776.030029,778.710022,772.890015,776.429993
10-05-16,779.309998,782.070007,775.650024,776.469971
10-06-16,779,780.47998,775.539978,776.859985
10-07-16,779.659973,779.659973,770.75,775.080017"""

df = pd.read_csv(io.StringIO(data))
df['Date'] = pd.to_datetime(df['Date'], format='%m-%d-%y')

plt.figure(figsize=(10, 6))
plt.plot(df['Date'], df['Open'], marker='o', label='Open')
plt.plot(df['Date'], df['High'], marker='s', label='High')
plt.plot(df['Date'], df['Low'], marker='^', label='Low')
plt.plot(df['Date'], df['Close'], marker='d', label='Close')

plt.title('Alphabet Inc. Financial Data (Oct 3, 2016 - Oct 7, 2016)', fontsize=14,
fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Price (USD)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(df['Date'], df['Date'].dt.strftime('%m-%d-%y'))
plt.tight_layout()
plt.show()
```

Output:



4. Write a Python program to plot two or more lines on the same plot with suitable legends for each line.

```
import matplotlib.pyplot as plt

days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

product_a_sales = [120, 140, 165, 130, 190, 210, 250]
product_b_sales = [90, 110, 105, 135, 150, 180, 220]
product_c_sales = [70, 85, 95, 110, 130, 160, 195]

plt.plot(days, product_a_sales, marker='o', color='blue', linestyle='-', linewidth=2,
label='Product A')
plt.plot(days, product_b_sales, marker='s', color='green', linestyle='--', linewidth=2,
label='Product B')
plt.plot(days, product_c_sales, marker='^', color='orange', linestyle='-.', linewidth=2,
label='Product C')

plt.title('Weekly Sales Comparison of Products', fontsize=14, fontweight='bold')
plt.xlabel('Days of the Week', fontsize=12)
```

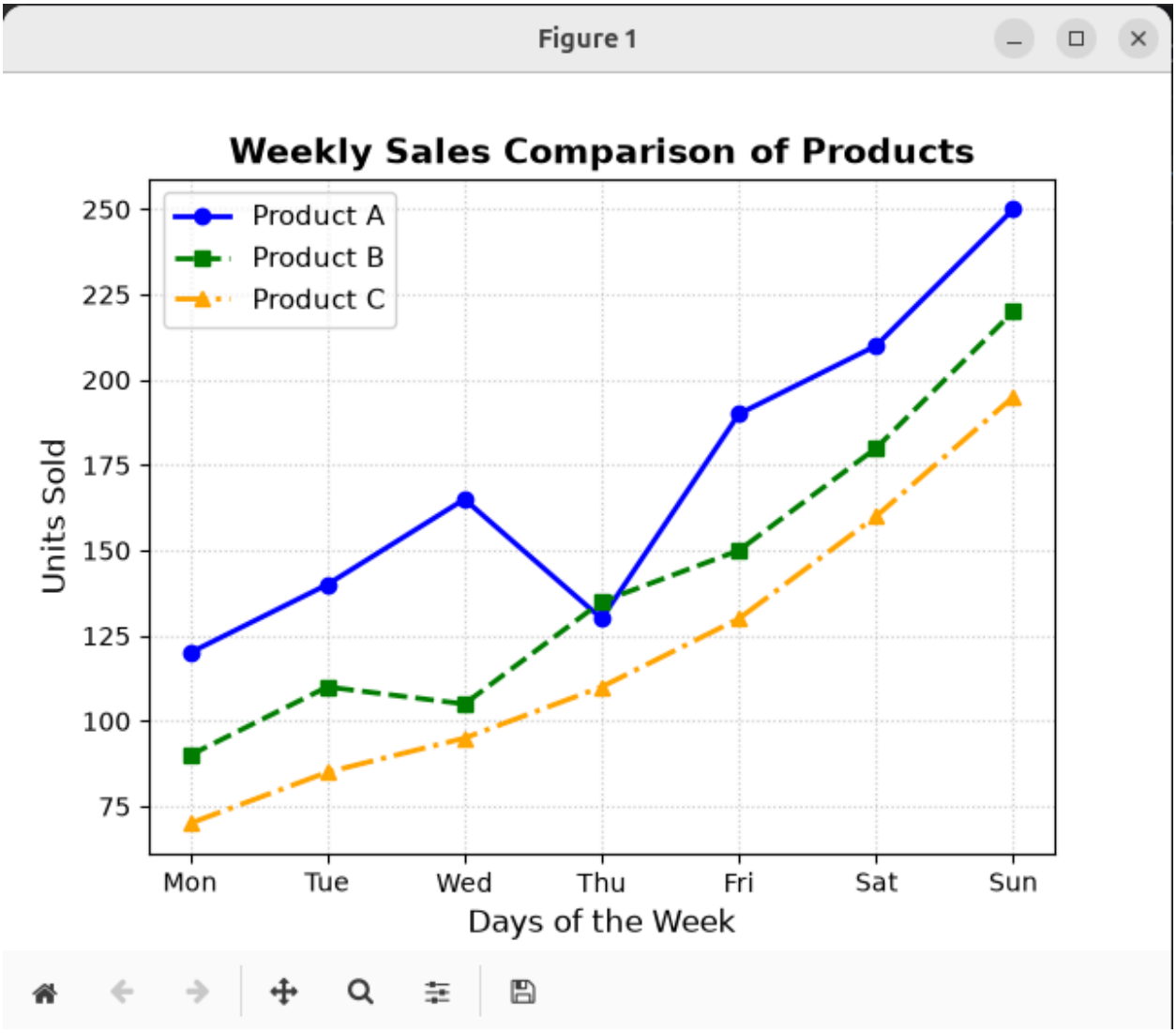
```
plt.ylabel('Units Sold', fontsize=12)

plt.legend(loc='upper left', fontsize=11)

plt.grid(True, linestyle=':', alpha=0.6)

plt.show()
```

Output:



5. Write a Pandas program to create and display a one-dimensional array-like object (Series) containing an array of data. Accept the values for the Series from the keyboard and display the Series.

```
import pandas as pd

user_input = input("Enter values separated by spaces: ")
data_list = user_input.split()

processed_list = []
for item in data_list:
    try:
        val = float(item)
        processed_list.append(int(val) if val.is_integer() else val)
    except ValueError:
        processed_list.append(item)

series = pd.Series(processed_list)

print("\nCreated Pandas Series:")
print(series)
```

Output:

```
Enter values separated by spaces: 10 20 30 40 50

Created Pandas Series:
0   10
1   20
2   30
3   40
4   50
dtype: int64
```

6. Write a Pandas program to convert a Pandas Series into a Python list and display the list and its type.

Hint: Use the tolist() function.

```
import pandas as pd

series = pd.Series([10, 20, 30, 40, 50])
```

```
python_list = series.tolist()

print("Python List:", python_list)
print("Type:", type(python_list))
```

Output:

```
Python List: [10, 20, 30, 40, 50]
Type: <class 'list'>
```

7. Write a Pandas program to add, subtract, multiply, and divide two Pandas Series. Accept the data through the keyboard and display the resultant Series.

Hint: Use the +, -, *, and / operators.

Sample Series: [2, 4, 6, 8, 10] and [1, 3, 5, 7, 9]

```
import pandas as pd

input1 = input("Enter values for First Series separated by spaces: ")
input2 = input("Enter values for Second Series separated by spaces: ")

series1 = pd.Series([float(x) for x in input1.split()])
series2 = pd.Series([float(x) for x in input2.split()])

print("\nAddition:")
print(series1 + series2)

print("\nSubtraction:")
print(series1 - series2)

print("\nMultiplication:")
print(series1 * series2)

print("\nDivision:")
print(series1 / series2)
```

Output:

```
Enter values for First Series separated by spaces: 2 4 6 8 10
Enter values for Second Series separated by spaces: 1 3 5 7 9
```

```
Addition:
0     3.0
1     7.0
2    11.0
3    15.0
4    19.0
dtype: float64

Subtraction:
0    1.0
1    1.0
2    1.0
3    1.0
4    1.0
dtype: float64

Multiplication:
0     2.0
1    12.0
2    30.0
3    56.0
4    90.0
dtype: float64

Division:
0    2.000000
1    1.333333
2    1.200000
3    1.142857
4    1.111111
dtype: float64
```

8. Write a Pandas program to compare the elements of two Pandas Series. Accept the data through the keyboard and display the results of ==, >, and < comparisons.

Hint: Use the ==, >, and < operators.

Sample Series: [2, 4, 6, 8, 10] and [1, 3, 5, 7, 10]

```
import pandas as pd

input1 = input("Enter values for First Series separated by spaces: ")
input2 = input("Enter values for Second Series separated by spaces: ")
```

```
series1 = pd.Series([float(x) for x in input1.split()])
series2 = pd.Series([float(x) for x in input2.split()])

print("\nEquals (==):")
print(series1 == series2)

print("\nGreater than (>):")
print(series1 > series2)

print("\nLess than (<):")
print(series1 < series2)
```

Output:

```
Enter values for First Series separated by spaces: 2 4 6 8 10
Enter values for Second Series separated by spaces: 1 3 5 7 10

Equals (==):
0    False
1    False
2    False
3    False
4     True
dtype: bool

Greater than (>):
0     True
1     True
2     True
3     True
4    False
dtype: bool

Less than (<):
0    False
1    False
2    False
3    False
4    False
dtype: bool
```

9. Write a Pandas program to create a dictionary containing empno, ename, and basic by accepting the values from the keyboard, and convert the dictionary into a Pandas Series.

Example dictionary: {'empno': 101, 'ename': 'ann', 'basic': 3000}

```
import pandas as pd

empno = int(input("Enter employee number: "))
ename = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))

emp_dict = {
   'empno': empno,
   'ename': ename,
   'basic': basic
}

emp_series = pd.Series(emp_dict)

print("\nPandas Series:")
print(emp_series)
```

Output:

```
Enter employee number: 101
Enter employee name: ann
Enter basic salary: 3000

Pandas Series:
empno     101
ename     ann
basic    3000.0
dtype: object
```

10. Write a Pandas program to accept 5 elements into a NumPy array and convert the NumPy array into a Pandas Series.

```
import numpy as np
import pandas as pd

user_input = input("Enter 5 elements separated by spaces: ")

np_array = np.array(user_input.split()[:5])
```

```
series = pd.Series(np_array)

print("\nPandas Series:")
print(series)
```

Output:

```
Enter 5 elements separated by spaces: 10 20 30 40 50

Pandas Series:
0   10
1   20
2   30
3   40
4   50
dtype: object
```

11. Write a Pandas program to accept 10 integers and calculate the mean and standard deviation of the data using a Pandas Series.

```
import pandas as pd

user_input = input("Enter 10 integers separated by spaces: ")

numbers = [int(x) for x in user_input.split()[:10]]
series = pd.Series(numbers)

mean_val = series.mean()
std_val = series.std()

print("\nPandas Series:")
print(series)
print("\nMean:", mean_val)
print("Standard Deviation:", std_val)
```

Output:

```
Enter 10 integers separated by spaces: 10 12 23 23 16 23 21 16 18 20

Pandas Series:
0   10
1   12
2   23
3   23
```

```
4   16
5   23
6   21
7   16
8   18
9   20
dtype: int64

Mean: 18.2
Standard Deviation: 4.63800723512926
```

12. Write a Pandas program to accept 10 integers and display the minimum, maximum, and sum of the values using a Pandas Series.

```
import pandas as pd

user_input = input("Enter 10 integers separated by spaces: ")

numbers = [int(x) for x in user_input.split()[:10]]
series = pd.Series(numbers)

min_val = series.min()
max_val = series.max()
sum_val = series.sum()

print("\nPandas Series:")
print(series)
print("\nMinimum:", min_val)
print("Maximum:", max_val)
print("Sum:", sum_val)
```

Output:

```
Enter 10 integers separated by spaces: 5 12 8 25 3 18 30 14 9 21

Pandas Series:
0    5
1   12
2    8
3   25
4    3
5   18
6   30
7   14
8    9
9   21
```

dtype: int64

Minimum: 3
Maximum: 30
Sum: 145

13. Write a Pandas program to accept a 15-digit number and display the count of each unique digit in the number.

```
import pandas as pd

user_input = input("Enter a 15-digit number: ").strip()

digits_list = list(user_input[:15])
series = pd.Series(digits_list)

digit_counts = series.value_counts()

print("\nCount of each unique digit:")
print(digit_counts)
```

Output:

```
Enter a 15-digit number: 987654321098765
Count of each unique digit:
9    2
8    2
7    2
6    2
5    2
4    1
3    1
2    1
1    1
0    1
Name: count, dtype: int64
```

14. Write a Pandas program to read a CSV file containing empno, ename, and basic details of 10 employees and display the first 3 rows of the DataFrame.

```
import pandas as pd
```

```
df = pd.read_csv("employees.csv")

print("First 3 rows of the DataFrame:")
print(df.head(3))
```

employees.csv

```
empno,ename,basic
101,Alice,50000
102,Bob,60000
103,Charlie,55000
104,David,62000
105,Eva,48000
106,Frank,53000
107,Grace,70000
108,Hank,45000
109,Ivy,68000
110,Jack,59000
```

Output:

```
First 3 rows of the DataFrame:
   empno    ename  basic
0    101    Alice  50000
1    102      Bob  60000
2    103  Charlie  55000
```

15. Write a Pandas program to select the ename and basic columns from the DataFrame populated from the CSV file.

```
import pandas as pd

df = pd.read_csv("employees.csv")

selected_columns = df[['ename', 'basic']]

print("Selected Columns (ename and basic):")
print(selected_columns)
```

employees.csv

```
ename,basic
Alice,50000
Bob,60000
```

Charlie,55000
David,62000
Eva,48000
Frank,53000
Grace,70000
Hank,45000
Ivy,68000
Jack,59000

Output:

Selected Columns (ename and basic):

	ename	basic
0	Alice	50000
1	Bob	60000
2	Charlie	55000
3	David	62000
4	Eva	48000
5	Frank	53000
6	Grace	70000
7	Hank	45000
8	Ivy	68000
9	Jack	59000

16. Write a Pandas program to select and display the rows where the basic salary is greater than 2000.

```
import pandas as pd

df = pd.read_csv("employees.csv")

filtered_df = df[df['basic'] > 2000]

print("Employees with basic salary > 2000:")
print(filtered_df)
```

employees.csv

```
ename,basic
Alice,50000
Bob,60000
Charlie,55000
David,62000
Eva,48000
Frank,53000
Grace,70000
```

Hank,45000
Ivy,68000
Jack,59000

Output:

Employees with basic salary > 2000:

	empno	ename	basic
0	101	Alice	50000
1	102	Bob	60000
2	103	Charlie	55000
3	104	David	62000
4	105	Eva	48000
5	106	Frank	53000
6	107	Grace	70000
7	108	Hank	45000
8	109	Ivy	68000
9	110	Jack	59000

17. Write a Pandas program to select and display the rows where the basic salary is missing (NaN).

```
import pandas as pd

df = pd.read_csv("employees.csv")

missing_basic_df = df[df['basic'].isna()]

print("Employees with missing basic salary (NaN):")
print(missing_basic_df)
```

employees.csv

```
empno,ename,basic
101,Alice,50000
102,Bob,
103,Charlie,55000
104,David,
105,Eva,48000
```

Output:

Employees with missing basic salary (NaN):

	empno	ename	basic
1	102	Bob	NaN
3	104	David	NaN

18. Write a Pandas program to accept an employee name and replace that name with "sam" in the DataFrame read from the CSV file. Display the updated DataFrame.

```
import pandas as pd

df = pd.read_csv("employees.csv")

target_name = input("Enter the employee name to replace: ")

df['ename'] = df['ename'].replace(target_name, 'sam')

print("\nUpdated DataFrame:")
print(df)
```

employees.csv

```
empno,ename,basic
101,Alice,50000
102,Bob,60000
103,Charlie,55000
104,David,62000
105,Eva,48000
```

Output:

```
Enter the employee name to replace: Bob
Updated DataFrame:
   empno    ename  basic
0    101    Alice  50000
1    102      sam  60000
2    103  Charlie  55000
3    104    David  62000
4    105      Eva  48000
```

19. Write a Pandas program to remove the basic column from the DataFrame and display the updated DataFrame.

```
import pandas as pd

df = pd.read_csv("employees.csv")
```

```
df_updated = df.drop(columns=['basic'])

print("DataFrame after removing the 'basic' column:")
print(df_updated)
```

employees.csv

```
empno,ename,basic

101,Alice,50000
102,Bob,60000
103,Charlie,55000
104,David,62000
105,Eva,48000
```

Output:

```
DataFrame after removing the 'basic' column:
   empno    ename
0    101    Alice
1    102      Bob
2    103  Charlie
3    104    David
4    105      Eva
```

20. Write a Pandas program to add a new column named commission, calculated as 2% of the basic salary, and store the updated DataFrame in a comma-separated CSV file named newfile.csv.

```
import pandas as pd

df = pd.read_csv("employees.csv")

df['commission'] = df['basic'] * 0.02

df.to_csv("newfile.csv", index=False)

print("Updated DataFrame:")
print(df)
print("\nDataFrame successfully saved to 'newfile.csv'")
```

employees.csv

```
empno,ename,basic
```

```
101,Alice,50000
102,Bob,60000
103,Charlie,55000
104,David,62000
105,Eva,48000
```

Output:

Updated DataFrame:

```
   empno    ename  basic  commission
0    101    Alice  50000      1000.0
1    102      Bob  60000      1200.0
2    103  Charlie  55000       1100.0
3    104    David  62000       1240.0
4    105      Eva  48000       960.0

DataFrame successfully saved to 'newfile.csv'
```

21. Write a Pandas program to add a new column named department to the employee CSV file. Assign 3 employees to the ACC department, 3 employees to the SLS department, and 4 employees to the PRD department. Load the CSV file into a DataFrame and display the department-wise count of employees.

```
import pandas as pd

df = pd.read_csv("employees.csv")

departments = ['ACC']*3 + ['SLS']*3 + ['PRD']*4
df['department'] = departments

df.to_csv("employees.csv", index=False)

dept_counts = df['department'].value_counts()

print("Department-wise Employee Count:")
print(dept_counts)
```

employees.csv

```
empno,ename,basic
101,Alice,50000
```

102,Bob,60000
103,Charlie,55000
104,David,62000
105,Eva,48000
106,Frank,53000
107,Grace,70000
108,Hank,45000
109,Ivy,68000
110,Jack,59000

Output:

Department-wise Employee Count:
department
PRD 4
ACC 3
SLS 3
Name: count, dtype: int64

22. Write a Pandas program to delete all rows from the DataFrame where the basic salary is less than 2000. Display the updated DataFrame.

```
import pandas as pd

df = pd.read_csv("employees.csv")

df = df[df['basic'] >= 2000]

print("Updated DataFrame (basic >= 2000):")
print(df)
```

employees.csv

empno,ename,basic
101,Alice,50000
102,Bob,1500
103,Charlie,55000
104,David,1800
105,Eva,48000

Output:

Updated DataFrame (basic >= 2000):

	empno	ename	basic
0	101	Alice	50000
2	103	Charlie	55000
4	105	Eva	48000